

# LAB ASSIGNMENT - 3.1

NAME: ISHA MADEEHA

HALL TICKET NO: 2303A51017

BATCH-01

TASK-1:

PROMPT: give python code for palindrome number or not

The screenshot shows the Visual Studio Code interface with a file named `task1.py` open. The code defines a function `is_palindrome_number(num)` that converts the number to a string, checks if it is equal to its reverse, and returns a boolean result. An example usage is provided with `number = 121`. The terminal output shows the command `python3 /Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py` being executed, resulting in the output `121 is a palindrome number.`

```
1  """
2  give python code for palindrome number or not
3  """
4  def is_palindrome_number(num):
5      # Convert the number to string
6      str_num = str(num)
7      # Check if the string is equal to its reverse
8      return str_num == str_num[::-1]
9  # Example usage
10 number = 121
11 if is_palindrome_number(number):
12     print(f"{number} is a palindrome number.")
13 else:
14     print(f"{number} is not a palindrome number.")
15 # Output: 121 is a palindrome number.
```

```
python3 /Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py
121 is a palindrome number.
```

The screenshot shows the Visual Studio Code interface with the same `task1.py` file. The example usage in the code has been changed to `number = 124`. The terminal output shows the command `python3 /Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py` being executed, resulting in the output `124 is not a palindrome number.`

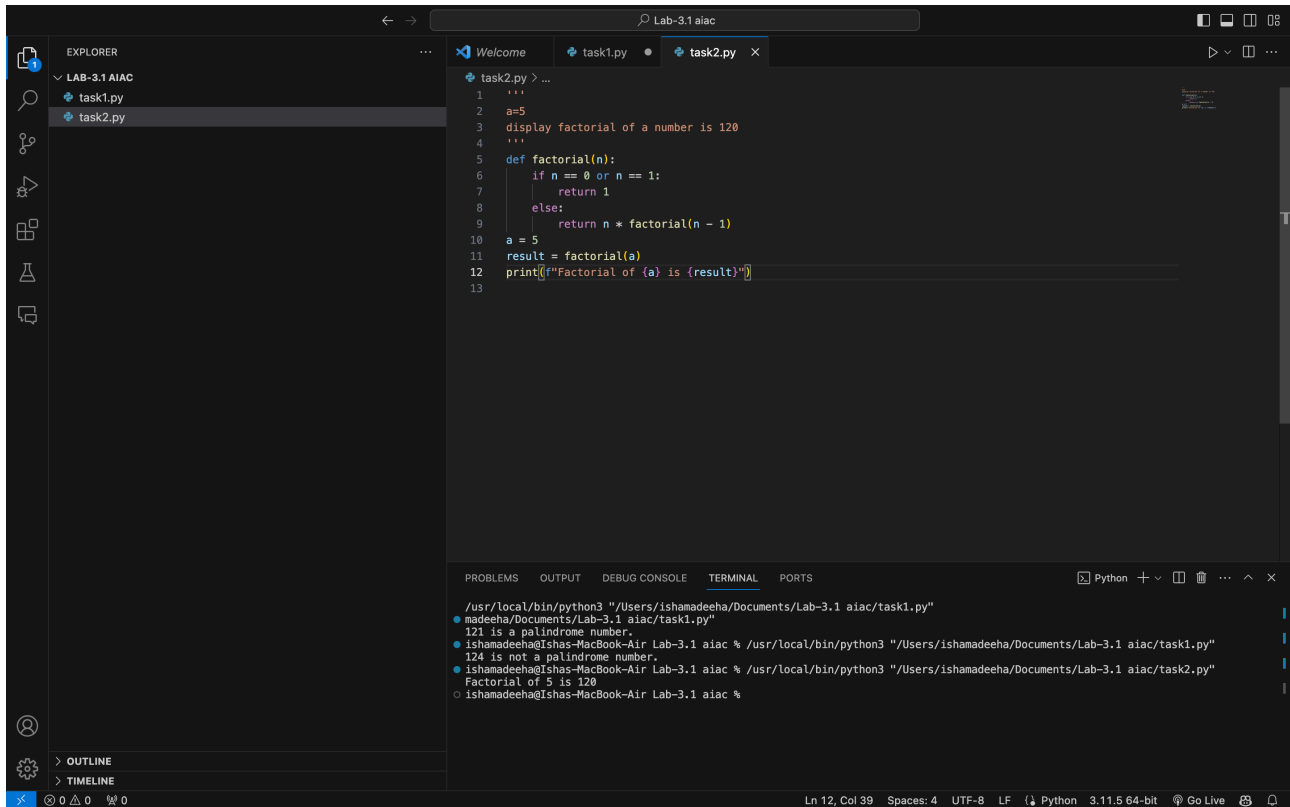
```
1  """
2  give python code for palindrome number or not
3  """
4  def is_palindrome_number(num):
5      # Convert the number to string
6      str_num = str(num)
7      # Check if the string is equal to its reverse
8      return str_num == str_num[::-1]
9  # Example usage
10 number = 124
11 if is_palindrome_number(number):
12     print(f"{number} is a palindrome number.")
13 else:
14     print(f"{number} is not a palindrome number.")
15
```

```
python3 /Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py
124 is not a palindrome number.
```

## TASK-2:

### PROMPT:

```
a=5  
display factorial of a number is 120
```



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'LAB-3.1 AIAC' containing two files: 'task1.py' and 'task2.py'. The code editor is open to 'task2.py', which contains the following Python code:

```
1  """  
2  a=5  
3  display factorial of a number is 120  
4  """  
5  def factorial(n):  
6      if n == 0 or n == 1:  
7          return 1  
8      else:  
9          return n * factorial(n - 1)  
10  
11 a = 5  
12 result = factorial(a)  
13 print(f"Factorial of {a} is {result}")
```

Below the code editor, the 'TERMINAL' panel shows the output of running the script:

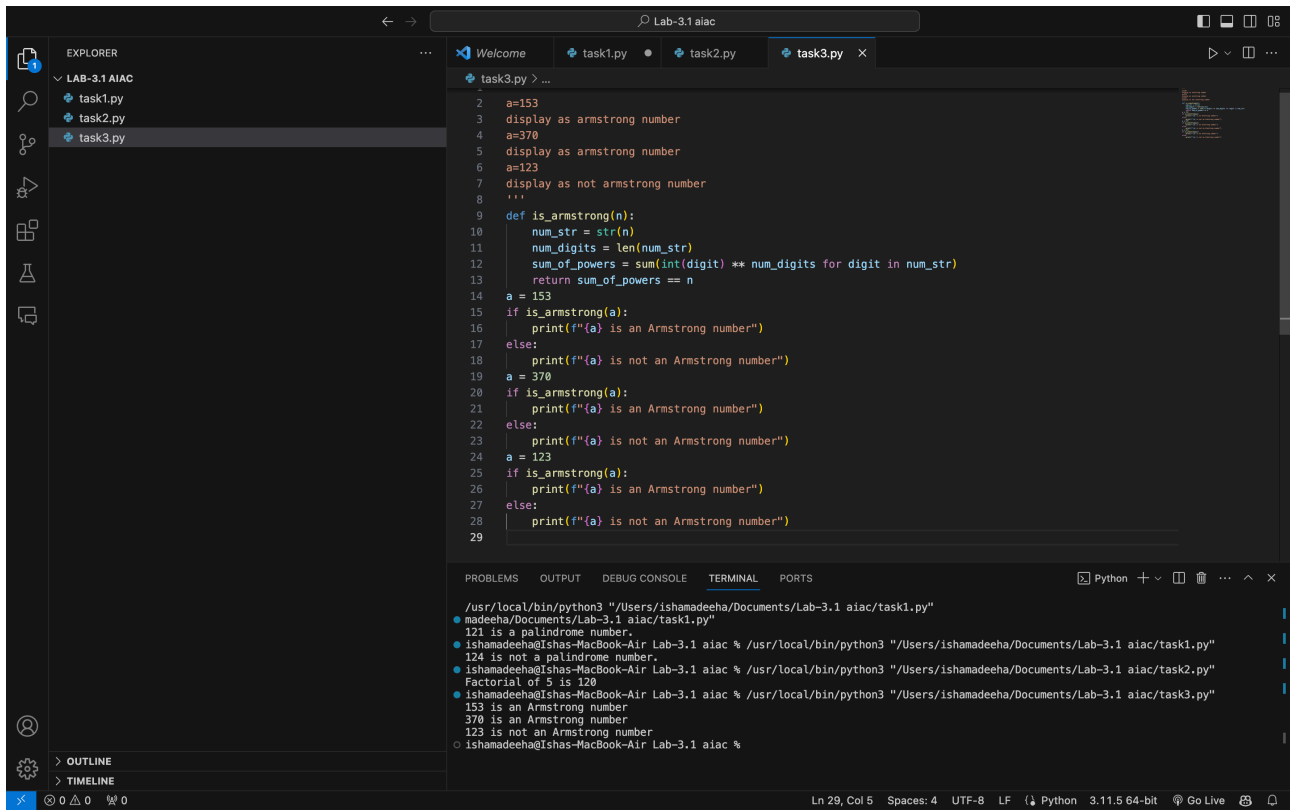
```
/usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py"  
madeeha/Documents/Lab-3.1 aiac/task1.py  
121 is a palindrome number.  
ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py"  
124 is not a palindrome number.  
ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task2.py"  
Factorial of 5 is 120  
ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac %
```

- Generated code is more structured and uses functions, making it cleaner and easier to reuse.
- A zero-shot solution is simpler, takes user input, and uses basic loops or math for beginners.

## TASK-3:

### PROMPT:

```
a=153  
display as armstrong number  
a=370  
display as armstrong number  
a=123  
display as not armstrong number
```



```
1 a=153
2 display as armstrong number
3 a=370
4 display as armstrong number
5 a=123
6 display as not armstrong number
7 ...
8
9 def is_armstrong(n):
10     num_str = str(n)
11     num_digits = len(num_str)
12     sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
13     return sum_of_powers == n
14
15 a = 153
16 if is_armstrong(a):
17     print(f"{a} is an Armstrong number")
18 else:
19     print(f"{a} is not an Armstrong number")
20 a = 370
21 if is_armstrong(a):
22     print(f"{a} is an Armstrong number")
23 else:
24     print(f"{a} is not an Armstrong number")
25 a = 123
26 if is_armstrong(a):
27     print(f"{a} is an Armstrong number")
28 else:
29     print(f"{a} is not an Armstrong number")
```

Terminal Output:

```
/usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py"
madeeha/Documents/Lab-3.1 aiac/task1.py
121 is a palindrome number.
ishamadeeha@ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py"
124 is not a palindrome number.
ishamadeeha@ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task2.py"
Factorial of 5 is 120
ishamadeeha@ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task3.py"
153 is an Armstrong number
370 is an Armstrong number
123 is not an Armstrong number
ishamadeeha@ishas-MacBook-Air Lab-3.1 aiac %
```

Using several examples (153, 370, 123) makes the function more accurate and reliable, but the code becomes repetitive because the same check is written many times.

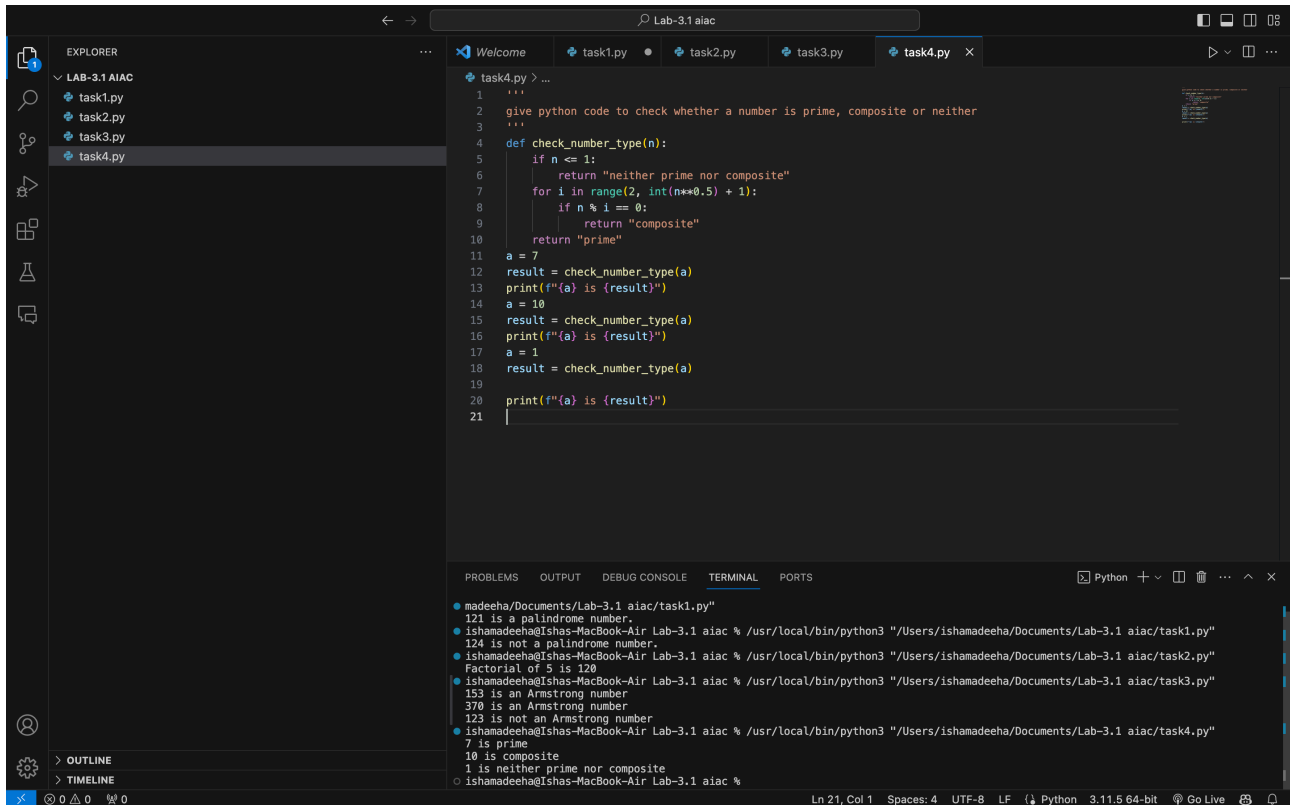
### Boundary & invalid tests:

- Test with 0 and 1 should return Armstrong number.
- Test with -153, 12.5, "abc" should be handled safely (otherwise the program may give wrong result).

TASK-4:

PROMPT:

give python code to check whether a number is prime, composite or neither



```
1  ...
2  give python code to check whether a number is prime, composite or neither
3  ...
4  def check_number_type(n):
5      if n <= 1:
6          return "neither prime nor composite"
7      for i in range(2, int(n**0.5) + 1):
8          if n % i == 0:
9              return "composite"
10         return "prime"
11
12 a = 7
13 result = check_number_type(a)
14 print(f"{a} is {result}")
15 a = 10
16 result = check_number_type(a)
17 print(f"{a} is {result}")
18 a = 1
19 result = check_number_type(a)
20 print(f"{a} is {result}")
21
```

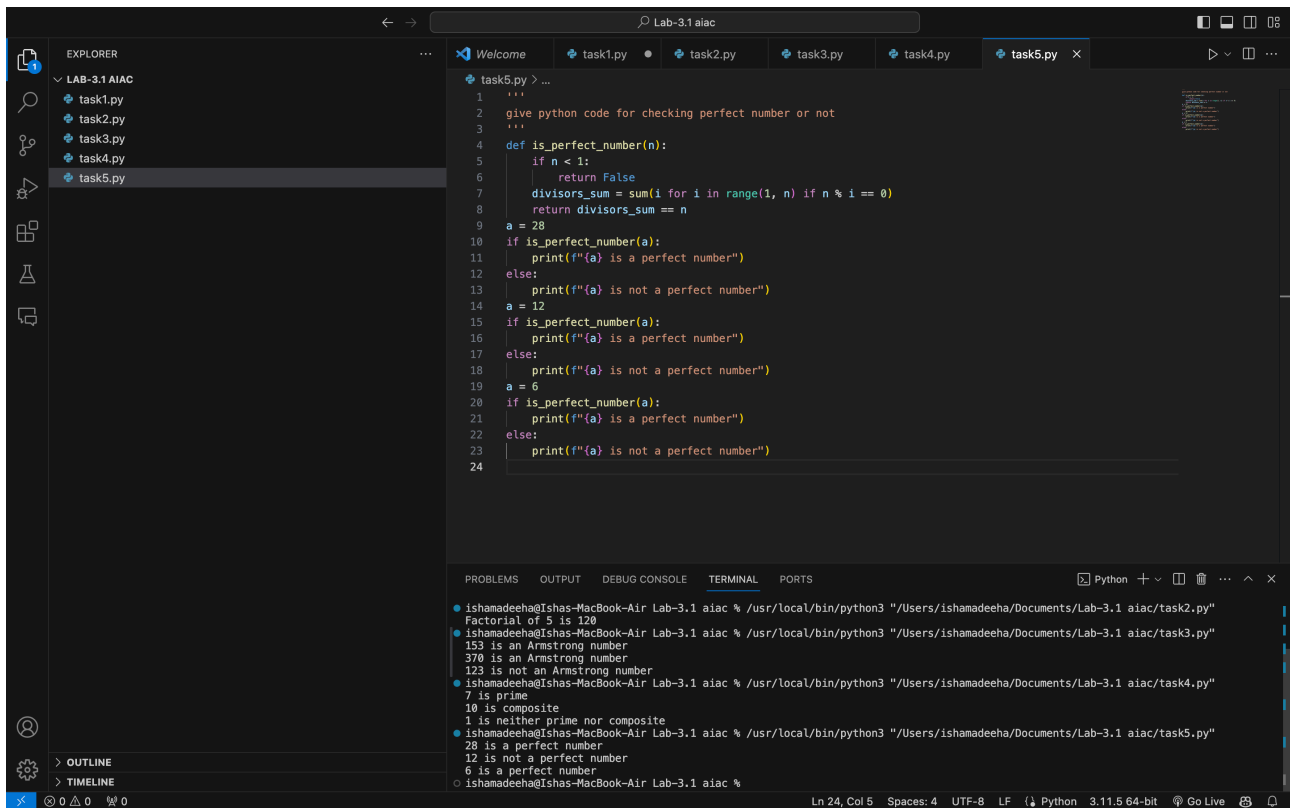
Terminal Output:

```
● madeeha/Documents/Lab-3.1 aiac/task1.py"
121 is a palindrome number.
● ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task1.py"
124 is not a palindrome number.
● ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task2.py"
Factorial of 5 is 120
● ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task3.py"
153 is an Armstrong number
370 is an Armstrong number
123 is not an Armstrong number
● ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac % /usr/local/bin/python3 "/Users/ishamadeeha/Documents/Lab-3.1 aiac/task4.py"
7 is prime
10 is composite
1 is neither prime nor composite
● ishamadeeha@Ishas-MacBook-Air Lab-3.1 aiac %
```

TASK-5:

PROMPT:

give python code for checking perfect number or not



## TASK-6:

### PROMPT:

```
a=8
display even number
a=15
display odd number
a=0
display even number
a=-4
display negative number
a=9.7
display as not as integer
```

